PRAKTIKUM SISTEM OPERASI

MODUL 9

Syscall

Oleh:

Candra Dinata - 2311104061

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO UNIVERSITAS TELKOM

2026

JURNAL

1. Syscall

```
#include <xinu.h>

pri16 chname(pid32 pid, pri16 newprio) {
    intmask mask;
    struct procent *prptr;
    pri16 oldprio;

    mask = disable();
    if (isbadpid(pid)) {
        restore(mask);
        return (pri16) SYSERR;
    }
    prptr = &proctab[pid];
    oldprio = prptr->prprio;
    prptr->prprio = newprio;
    kprintf("\n\nNew Syscall is easy\n\n");
    restore(mask);
    return oldprio;
}
```

```
xinu@xinu-develop-end: ~/xinu_m9/compile

--       --       -----       -       -       ----
-----------------------------------------


Welcome to Xinu!


xsh $ ps
Pid Name              State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull            ready    0    0 0x005FDFFC 0x00FFFEE4     8192
  1 ipout             wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin             wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process      recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell             recv    50    4 0x005D7FFC 0x005D79AC     8192
  6 ps                curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $ uptime
Xinu has been up  1 second(s)


New Syscall is easy


New Syscall is easy

xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

Pada praktikum ini dilakukan modifikasi kernel sistem operasi Xinu dengan menambahkan sebuah **system call (syscall)** baru bernama chname. System call merupakan layanan yang disediakan kernel agar program dapat meminta operasi tertentu secara langsung kepada sistem. Pada kode ini, syscall chname digunakan untuk **mengubah prioritas suatu proses** yang sedang aktif di dalam tabel proses Xinu.

Fungsi chname(pid32 pid, pri16 newprio) menerima dua parameter yaitu pid sebagai identitas proses yang akan diubah, dan newprio sebagai nilai prioritas baru yang akan diberikan. Di awal fungsi dibuat beberapa variabel seperti mask untuk menyimpan status interrupt, *prptr sebagai pointer menuju struktur proses, serta oldprio untuk menyimpan prioritas lama proses sebelum diubah. Selanjutnya sistem menjalankan disable() untuk menonaktifkan interrupt sementara agar proses perubahan data kernel berlangsung aman tanpa gangguan proses lain.

Setelah itu dilakukan pengecekan melalui isbadpid(pid) untuk memastikan PID yang dimasukkan valid. Jika PID tidak valid maka interrupt dikembalikan dengan restore(mask) dan fungsi akan mengembalikan nilai SYSERR sebagai tanda error. Namun apabila PID valid, pointer prptr diarahkan ke proctab[pid] yaitu tabel proses milik Xinu. Nilai prioritas lama proses disimpan ke variabel oldprio, kemudian prioritas proses diubah dengan perintah prptr->prprio = newprio;. Setelah perubahan berhasil, program menampilkan pesan "New Syscall is easy" sebagai bukti bahwa syscall telah dipanggil dan dijalankan.

Di bagian akhir, interrupt dikembalikan ke kondisi semula menggunakan restore(mask) lalu fungsi mengembalikan oldprio. Pengembalian nilai prioritas lama ini berguna agar pengguna dapat mengetahui berapa nilai prioritas proses sebelum dilakukan perubahan. Dengan demikian syscall ini tidak hanya melakukan modifikasi data proses, tetapi juga memberikan informasi hasil perubahan kepada pemanggil.

Berdasarkan output terminal, setelah kernel Xinu dijalankan muncul perintah ps yang menampilkan daftar proses aktif seperti prnull, ipout, netin, Main process, shell, dan ps. Selanjutnya saat syscall dipanggil, sistem berhasil menampilkan tulisan **"New Syscall is easy"** pada layar. Hal ini menandakan bahwa syscall buatan telah berhasil ditambahkan ke kernel Xinu, berhasil dikompilasi, dan dapat dieksekusi dengan baik tanpa error. Dengan adanya percobaan ini dapat dipahami bahwa Xinu memungkinkan pengguna untuk melakukan modifikasi layanan kernel secara langsung, khususnya pada manajemen proses seperti perubahan prioritas proses.

2. syscall chprio

Open ▾ chprio.c ~/xinu_m9/system Save

```c
/* chprio.c - chprio */

#include <xinu.h>

/*------------------------------------------------------------------------
 *  chprio  -  Change the scheduling priority of a process
 *------------------------------------------------------------------------
 */
syscall chprio(
          pid32         pid,            /* ID of process to change      */
          pri16         newprio         /* New priority                 */
        )
{
        intmask mask;                   /* Saved interrupt mask         */
        struct  procent *prptr;         /* Ptr to process's table entry */
        pri16   oldprio;                /* Priority to return           */

        mask = disable();
        if (isbadpid(pid)) {
                restore(mask);
                return SYSERR;
        }

        if (newprio <= 0) {
                restore(mask);
                return SYSERR;
        }
```

C ▾ Tab Width: 8 ▾ Ln 17, Col 1 ▾ INS

Modifikasi terhadap file sistem xinu/system/chprio.c yang berisi implementasi **system call chprio**. Syscall ini digunakan untuk **mengubah prioritas penjadwalan (scheduling priority) suatu proses** pada sistem operasi Xinu. Prioritas proses berfungsi sebagai penentu tingkat kepentingan proses saat scheduler memilih proses mana yang akan dijalankan terlebih dahulu. Semakin besar nilai prioritas, maka semakin tinggi peluang proses tersebut memperoleh jatah CPU.

Fungsi ini dideklarasikan sebagai:

syscall chprio(pid32 pid, pri16 newprio)

Parameter pid menunjukkan **ID proses** yang akan diubah prioritasnya, sedangkan newprio menunjukkan **nilai prioritas baru** yang akan diberikan. Di dalam fungsi juga terdapat beberapa variabel pendukung yaitu mask untuk menyimpan status interrupt, *prptr sebagai pointer menuju entri proses dalam process table, dan oldprio untuk menyimpan prioritas lama sebelum diubah.

Langkah pertama yang dilakukan program adalah menjalankan:

mask = disable();

Perintah ini digunakan untuk menonaktifkan interrupt sementara, sehingga perubahan data pada kernel dapat berlangsung aman tanpa terganggu proses lain.

Validasi Input PID (ID Proses)

Sesuai instruksi praktikum, input pid harus dipastikan berupa angka yang berada pada rentang **0 sampai NPROC**, di mana NPROC adalah jumlah maksimum proses yang dapat ditampung oleh Xinu. Untuk melakukan pengecekan ini digunakan fungsi bawaan:

```
if (isbadpid(pid)) {
   restore(mask);
   return SYSERR;
}
```

Fungsi isbadpid(pid) akan bernilai benar apabila PID:

- kurang dari 0,
- melebihi NPROC,
- atau tidak menunjuk ke proses yang valid.

Jika kondisi tersebut terjadi maka:

1. interrupt dikembalikan dengan restore(mask),
2. syscall dibatalkan,
3. fungsi mengembalikan SYSERR sebagai tanda kesalahan.

Artinya sistem tidak akan memproses perubahan prioritas apabila ID proses yang dimasukkan salah. Validasi ini penting untuk mencegah kernel mengakses alamat process table yang tidak semestinya.

Validasi Input Prioritas Baru

Selain PID, parameter newprio juga harus divalidasi. Berdasarkan ketentuan, prioritas harus berupa **bilangan positif** atau lebih besar dari nol. Oleh karena itu pada source code ditambahkan pengecekan:

```
if (newprio <= 0) {
   restore(mask);
   return SYSERR;
}
```

Kondisi ini memastikan bahwa nilai prioritas:

- tidak boleh nol,
- tidak boleh negatif.

Jika pengguna memasukkan prioritas yang tidak valid, maka syscall langsung dihentikan dan mengembalikan SYSERR. Validasi ini dilakukan agar sistem penjadwalan Xinu tetap berjalan normal, karena prioritas nol atau negatif tidak sesuai dengan aturan scheduling.

Proses Perubahan Prioritas

Apabila kedua input lolos validasi, maka program akan melanjutkan ke proses perubahan prioritas dengan mengambil data proses dari tabel proses:

```
prptr = &proctab[pid];
oldprio = prptr->prprio;
prptr->prprio = newprio;
```

Penjelasannya:

- prptr = &proctab[pid]; → pointer diarahkan ke data proses sesuai PID.
- oldprio = prptr->prprio; → menyimpan prioritas lama.
- prptr->prprio = newprio; → mengganti prioritas lama dengan prioritas baru.

Dengan langkah ini, data proses di kernel berhasil diperbarui sesuai input pengguna.

Mengembalikan Kondisi Sistem

Setelah perubahan selesai dilakukan, interrupt dikembalikan seperti semula dengan:

```
restore(mask);
return oldprio;
```

Perintah restore(mask) mengaktifkan kembali interrupt, sedangkan return oldprio berfungsi untuk mengembalikan nilai prioritas lama sebagai informasi bahwa perubahan telah berhasil dilakukan

3.

```
xinu@xinu-develop-end: ~/xinu_m9/compile
File  Edit  View  Search  Terminal  Help
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFEE4     8192
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  6 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $ uptime


New Syscall is easy

Xinu has been up  3 second(s)


New Syscall is easy

xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFEE4     8192
  1 ipout            wait    20    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    33    4 0x005D7FFC 0x005D79AC     8192
  8 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

Setelah system call chprio berhasil dimodifikasi dan dikompilasi pada kernel Xinu, langkah berikutnya adalah melakukan pengujian menggunakan beberapa perintah shell yaitu ps, uptime, dan ps kembali. Pengujian ini bertujuan untuk membuktikan bahwa syscall chprio benar-benar mampu mengubah prioritas proses yang dipilih, khususnya pada proses dengan **PID = 5**.

Menjalankan Perintah ps Sebelum Syscall Dipanggil

Langkah awal dilakukan dengan mengetik:

xsh $ ps

Perintah ps berfungsi untuk menampilkan seluruh daftar proses yang sedang aktif pada sistem Xinu beserta informasi penting seperti:

- PID proses,
- nama proses,
- state proses,
- prioritas proses,
- parent PID,
- stack base,
- stack pointer,
- dan ukuran stack.

Berdasarkan output pertama pada gambar, terlihat bahwa proses dengan:

- **PID 5 = shell**
- memiliki nilai **prioritas 50**

Selain itu proses lain seperti:

- ipout memiliki prioritas 500,
- Main process memiliki prioritas 20,
- ps memiliki prioritas 20.

Nilai prioritas ini merupakan kondisi awal sebelum syscall chprio dijalankan.

Menjalankan Perintah uptime

Langkah berikutnya adalah menjalankan:
xsh $ uptime
Perintah uptime pada Xinu digunakan untuk menampilkan lama waktu sistem telah aktif sejak booting. Pada output terlihat tulisan:
Xinu has been up 3 second(s)
Namun pada praktikum ini, perintah uptime juga telah dimodifikasi sehingga di dalam proses eksekusinya memanggil syscall chprio. Hal ini dibuktikan dengan munculnya pesan:
New Syscall is easy
Pesan tersebut menandakan bahwa syscall chprio berhasil dieksekusi ketika uptime dijalankan. Pada saat itulah kernel melakukan perubahan prioritas terhadap proses yang telah ditentukan.
Dalam kasus ini syscall diprogram untuk mengubah prioritas:

- proses ipout (PID 1) dari 500 menjadi 20,
- dan proses shell (PID 5) dari 50 menjadi 33.

Karena shell adalah proses yang sedang digunakan pengguna, maka perubahan prioritas PID 5 menjadi fokus utama pengamatan sesuai instruksi soal.

3. Menjalankan Perintah ps Setelah Syscall Dipanggil

Setelah uptime selesai dijalankan, pengguna kembali mengetik:
xsh $ ps
Tujuan langkah ini adalah membandingkan daftar proses setelah syscall chprio dieksekusi.
Pada output kedua terlihat bahwa:

- proses dengan **PID 5 (shell)** yang sebelumnya memiliki prioritas **50**
- kini berubah menjadi **33**

Selain itu:

- proses ipout yang sebelumnya 500 juga berubah menjadi 20.

Perubahan ini membuktikan bahwa syscall chprio telah berhasil memodifikasi nilai prprio pada tabel proses proctab.

Analisis Perubahan Prioritas PID = 5

Sesuai instruksi soal, fokus utama pengamatan adalah proses dengan **ID = 5**.
Sebelum syscall dijalankan:

PID	Nama Proses	Prioritas
5	shell	50

Setelah uptime memanggil syscall:

PID	Nama Proses	Prioritas
5	shell	33

Artinya prioritas proses shell berhasil berubah dari **50 menjadi 33**.

Perubahan ini menunjukkan bahwa:

- syscall menerima PID yang valid yaitu 5,
- syscall menerima prioritas baru yang valid yaitu 33,
- kemudian field prprio pada proctab[5] berhasil diperbarui.

Dengan demikian hasil ps kedua sesuai dengan yang diharapkan pada soal, yaitu **prioritas proses dengan ID = 5 sudah berubah**.

Setelah syscall chprio berhasil dijalankan dengan input yang benar, dilakukan pengujian kembali menggunakan input yang salah untuk memastikan validasi pada program bekerja dengan baik. Pengujian pertama dilakukan dengan mengubah pemanggilan chprio(5,33) menjadi chprio(5,-3) pada file xsh_uptime.c. Pada kondisi ini PID 5 masih valid, namun nilai prioritas -3 tidak memenuhi syarat karena prioritas harus berupa bilangan positif. Saat program dijalankan, syscall akan mendeteksi bahwa newprio <= 0 sehingga fungsi langsung mengembalikan SYSERR dan perubahan prioritas dibatalkan. Hal ini dibuktikan dengan perintah ps yang menunjukkan bahwa prioritas proses shell tidak berubah.
Pengujian kedua dilakukan dengan mengubah pemanggilan menjadi chprio(3000,3). Pada percobaan ini nilai prioritas 3 valid, tetapi PID 3000 tidak valid karena berada di luar batas jumlah proses maksimum NPROC. Melalui pengecekan isbadpid(pid), sistem mendeteksi bahwa ID proses tersebut salah sehingga syscall kembali dibatalkan dan tidak melakukan perubahan pada tabel proses. Setelah dicek menggunakan perintah ps, terlihat bahwa prioritas proses tetap sama seperti sebelumnya. Dari kedua pengujian tersebut dapat disimpulkan bahwa syscall chprio yang telah dimodifikasi berhasil menolak argumen yang salah, baik berupa prioritas negatif maupun PID yang tidak valid, sehingga prioritas proses tidak berubah ketika input tidak sesuai.